Experiment No. 1: Lab Report & Implementation

Lab Document Header

- **PRN:** 23510010
- **Batch:** B5
- **Experiment No:** 1
- **Title:** Introduction to HPC Environment and Shared Memory Parallel Programming
- **Objective:** To understand the HPC environment and implement basic programs using OpenMP.
- **Problem Statement:** High Performance Computing (HPC) involves the use of parallel processing techniques to solve computationally intensive problems. Study basics of OpenMP for shared memory parallel programming using CPU threads.

Task 1: HPC Environment Setup and System Configuration

1. System Configuration

Based on the `lscpu` terminal output from the **Acer Nitro V (ANV16-71)**, the hardware and system configuration is recorded below:

Parameter	Specification (From lscpu)
Processor Model	Intel(R) Core(TM) i7-14650HX (14th Gen Raptor Lake-HX)
Number of CPU Cores	**16 Cores** (8 Performance-cores + 8 Efficient-cores)
Number of Logical Processors	**24 Threads** (0–23 online CPUs)
CPU Frequency	**5200.0000 MHz** Max Turbo / **800.0000 MHz** Min

Cache Architecture	L1d: 640 KiB, L1i: 768 KiB, L2: 24 MiB, L3: 30 MiB
RAM Size	**16 GB** DDR5
Operating System	Linux (Ubuntu x86_64)
SIMD Instruction Support	SSE, SSE2, SSE3, SSE4_1, SSE4_2, AVX, AVX2, FMA (SIMD Width = 8 for Single Precision)

2. Verification of Linux Environment & Tools

```
raren@raren-Nitro-ANV16-71:~$ gcc --version
Command 'gcc' not found, but can be installed with:
sudo apt install gcc
raren@raren-Nitro-ANV16-71:~$ sudo apt update
[sudo: authenticate] Password:
Get:1 https://dl.google.com/linux/chrome-stable/deb stable InRelease [2,548 B]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:3 http://in.archive.ubuntu.com/ubuntu resolute InRelease
Get:4 https://dl.google.com/linux/chrome-stable/deb stable/main amd64 Packages [1,413 B]
Get:5 http://in.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [341 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [32.8 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [150 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.2 kB]
Hit:10 http://in.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:11 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [402 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [62.2 kB]
Get:13 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [223 kB]
Get:14 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [72.7 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [161 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe Icons (48x48) [45.9 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe Icons (64x64) [69.8 kB]
Fetched 1,883 kB in 10s (181 kB/s)
271 packages can be upgraded. Run 'apt list --upgradable' to see them.
raren@raren-Nitro-ANV16-71:~$ sudo apt install build-essentials
Error: Unable to locate package build-essentials
raren@raren-Nitro-ANV16-71:~$ sudo apt install build-essential
The followlng packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14          linux-modules-7.0.0-14-generic
  linux-headers-7.0.0-14-generic  linux-tools-7.0.0-14
  linux-image-unsigned-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.

Installing:
  build-essential

Installing dependencies:
  bzip2                 gcc-x86-64-linux-gnu      libgcc-15-dev
  dpkg-dev              libalgorithm-diff-perl    libhwasan0
  fakeroot              libalgorithm-diff-xs-perl  libitm1
  g++                   libalgorithm-merge-perl   liblsan0
  g++-15                libasan8                  libquadmath0
  g++-15-x86-64-linux-gnu  libcc1-0               libstdc++-15-dev
  g++-x86-64-linux-gnu  libcrypt-dev              libtsan2
  gcc                   libdpkg-perl              libubsan1
  gcc-15                libfakeroot               lto-disabled-list
  gcc-15-x86-64-linux-gnu  libfile-fcntllock-perl  make

Suggested packages:
  bzip2-doc             autoconf         gcc-15-locales
  debian-keyring        automake         gdb-x86-64-linux-gnu
  debian-tag2upload-keyring  libtool     git
  g++-multilib          flex             bzr
  g++-15-multilib       bison            libstdc++-15-doc
  gcc-15-doc            gcc-doc          make-doc
  gcc-multilib          gcc-15-multilib

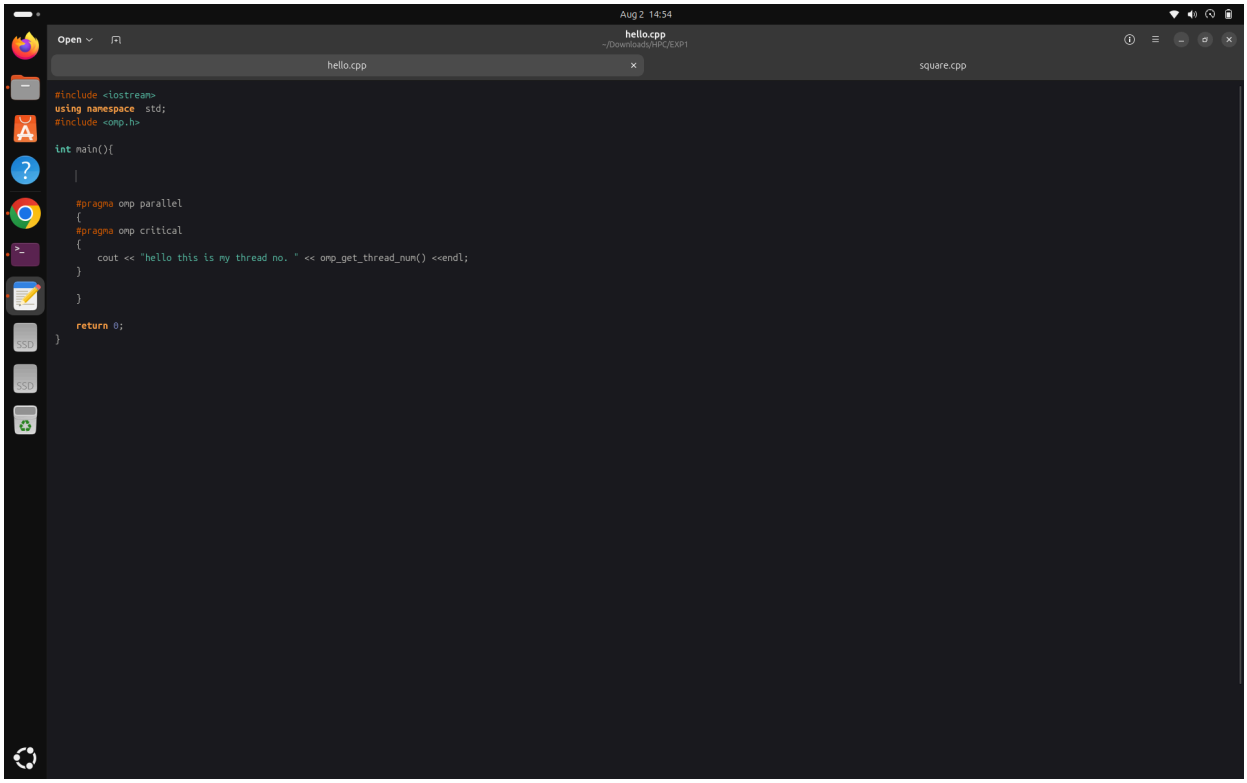
Summary:
  Upgrading: 0, Installing: 31, Removing: 0, Not Upgrading: 271
  Download size: 50.8 MB
  Space needed: 205 MB / 35.5 GB available
```

```
Processing triggers for libc-bin (2.43-2ubuntu2)…
raren@raren-Nitro-ANV16-71:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

raren@raren-Nitro-ANV16-71:~$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             39 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      24
  On-line CPU(s) list:       0-23
Vendor ID:                   GenuineIntel
  Model name:                Intel(R) Core(TM) i7-14650HX
    CPU family:              6
    Model:                   183
    Thread(s) per core:      2
    Core(s) per socket:      16
    Socket(s):               1
    Stepping:                1
    CPU(s) scaling MHz:      50%
    CPU max MHz:             5200.0000
    CPU min MHz:             800.0000
    BogoMIPS:                4838.40
    Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                             e mca cmov pat pse36 clflush dts acpi mmx fxsr sse
                             sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm cons
                             tant_tsc art arch_perfmon pebs bts rep_good nopl xt
                             opology nonstop_tsc cpuid aperfmperf tsc_known_freq
                              pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ss
                             se3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2ap
                             ic movbe popcnt tsc_deadline_timer aes xsave avx f1
                             6c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb
                              ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow flex
                             priority ept vpid ept_ad fsgsbase tsc_adjust bmi1 a
                             vx2 smep bmi2 erms invpcid rdseed adx smap clflusho
                             pt clwb intel_pt sha_ni xsaveopt xsavec xgetbv1 xsa
                             ves split_lock_detect user_shstk avx_vnni dtherm id
                             a arat pln pts hwp hwp_notify hwp_act_window hwp_ep
                             p hwp_pkg_req hfi vnmi umip pku ospke waitpkg gfni
                             vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_cle
                             ar serialize arch_lbr ibt flush_l1d arch_capabiliti
                             es
Virtualization features:
  Virtualization:            VT-x
Caches (sum of all):
  L1d:                       640 KiB (16 instances)
  L1i:                       768 KiB (16 instances)
  L2:                        24 MiB (10 instances)
  L3:                        30 MiB (1 instance)
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-23
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
```

3. Basic Verification Program: Multi-Threaded "Hello World"

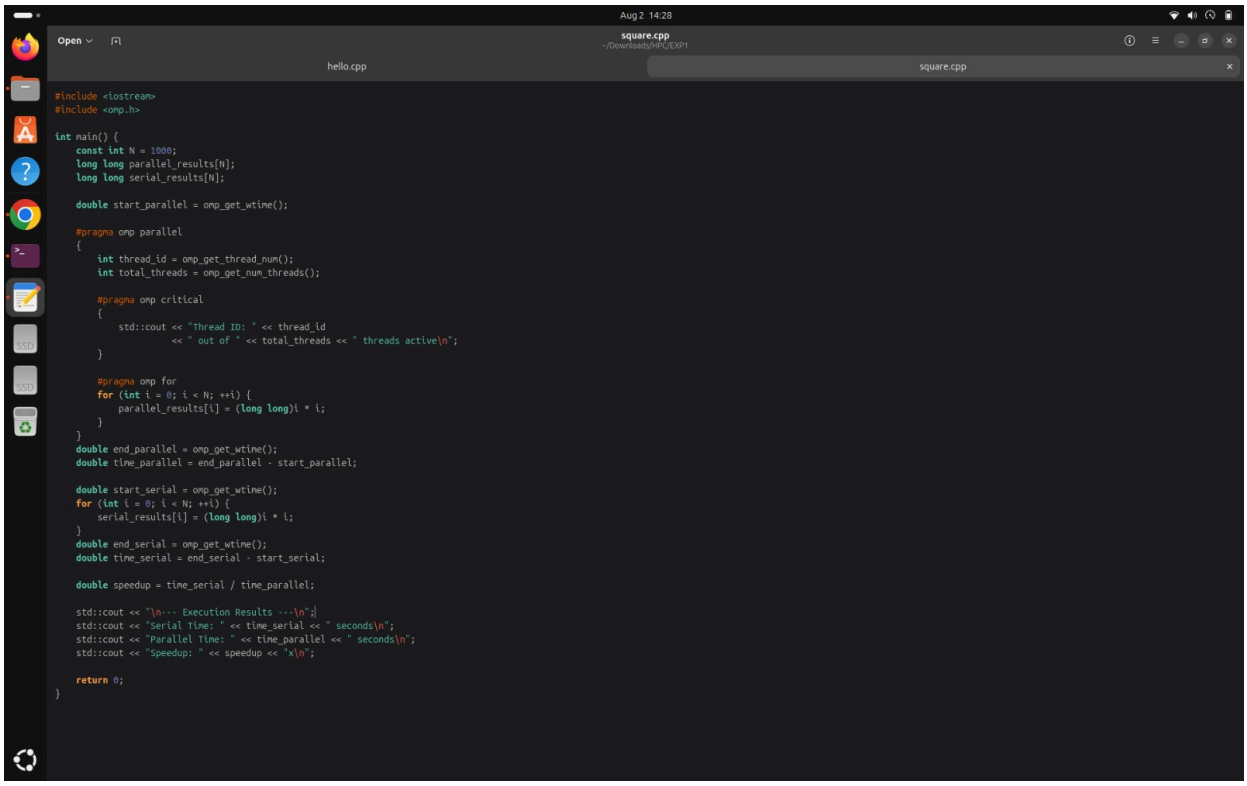
Source Code (hello.cpp):

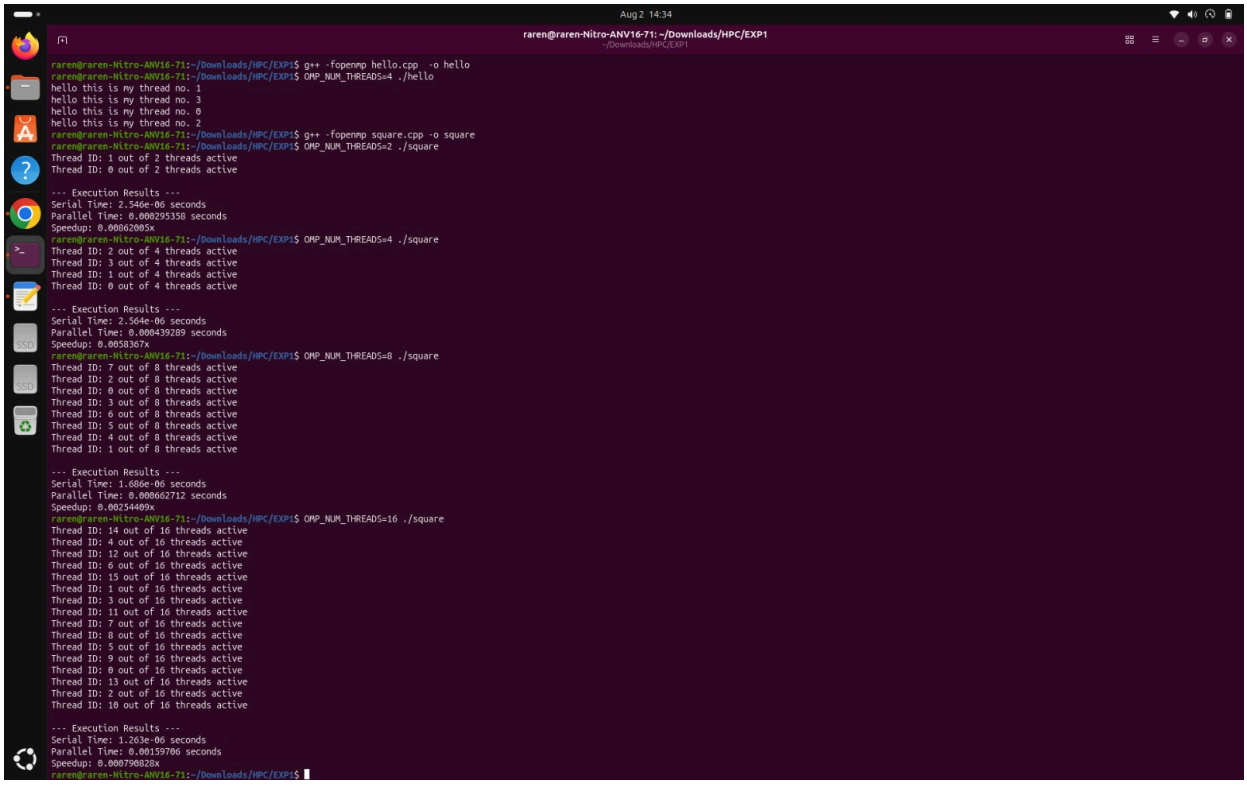



```
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ ./hello
hello this is my thread no. hello this is my thread no. 0
hello this is my thread no. 4
hello this is my thread no. 3
hello this is my thread no. 5
hello this is my thread no. 8
hello this is my thread no. 7
hello this is my thread no. 2
hello this is my thread no. hello this is my thread no. hello this is my thread no. 15
hello this is my thread no. 10
hello this is my thread no. hello this is my thread no. hello this is my thread no. hello this is my thread no. 1312
11

9
1
6
14
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ g++ -fopenmp hello.cpp -o hello
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ ./hello
hello this is my thread no. 0
hello this is my thread no. 5
hello this is my thread no. 8
hello this is my thread no. 14
hello this is my thread no. 15
hello this is my thread no. 10
hello this is my thread no. 2
hello this is my thread no. 7
hello this is my thread no. 6
hello this is my thread no. 11
hello this is my thread no. 1
hello this is my thread no. 3
hello this is my thread no. 4
hello this is my thread no. 12
hello this is my thread no. 9
hello this is my thread no. 13
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$
```

Task 2: OpenMP Thread Creation & Performance Analysis


Aug 2 14:28
Open
square.cpp
~/Downloads/HPC/EXP1
hello.cpp
square.cpp
#include <iostream>
#include <omp.h>
int main() {
const int N = 1000;
long long parallel_results[N];
long long serial_results[N];
double start_parallel = omp_get_wtime();
#pragma omp parallel
{
int thread_id = omp_get_thread_num();
int total_threads = omp_get_num_threads();
#pragma omp critical
{
std::cout << "Thread ID: " << thread_id
<< " out of " << total_threads << " threads active\n";
}
#pragma omp for
for (int i = 0; i < N; ++i) {
parallel_results[i] = (long long)i * i;
}
}
double end_parallel = omp_get_wtime();
double time_parallel = end_parallel - start_parallel;
double start_serial = omp_get_wtime();
for (int i = 0; i < N; ++i) {
serial_results[i] = (long long)i * i;
}
double end_serial = omp_get_wtime();
double time_serial = end_serial - start_serial;
double speedup = time_serial / time_parallel;
std::cout << "\n--- Execution Results ---\n";
std::cout << "Serial Time: " << time_serial << " seconds\n";
std::cout << "Parallel Time: " << time_parallel << " seconds\n";
std::cout << "Speedup: " << speedup << "x\n";
return 0;
}


Aug 2 14:34
raren@raren-Nitro-ANV16-71: ~/Downloads/HPC/EXP1
~/Downloads/HPC/EXP1
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ g++ -fopenmp hello.cpp -o hello
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ OMP_NUM_THREADS=4 ./hello
hello this is my thread no. 1
hello this is my thread no. 3
hello this is my thread no. 0
hello this is my thread no. 2
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ g++ -fopenmp square.cpp -o square
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ OMP_NUM_THREADS=2 ./square
Thread ID: 1 out of 2 threads active
Thread ID: 0 out of 2 threads active
--- Execution Results ---
Serial Time: 2.546e-06 seconds
Parallel Time: 0.000295358 seconds
Speedup: 0.00862005x
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ OMP_NUM_THREADS=4 ./square
Thread ID: 2 out of 4 threads active
Thread ID: 3 out of 4 threads active
Thread ID: 1 out of 4 threads active
Thread ID: 0 out of 4 threads active
--- Execution Results ---
Serial Time: 2.564e-06 seconds
Parallel Time: 0.000439289 seconds
Speedup: 0.0058367x
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ OMP_NUM_THREADS=8 ./square
Thread ID: 7 out of 8 threads active
Thread ID: 2 out of 8 threads active
Thread ID: 0 out of 8 threads active
Thread ID: 3 out of 8 threads active
Thread ID: 6 out of 8 threads active
Thread ID: 5 out of 8 threads active
Thread ID: 4 out of 8 threads active
Thread ID: 1 out of 8 threads active
--- Execution Results ---
Serial Time: 1.686e-06 seconds
Parallel Time: 0.000662712 seconds
Speedup: 0.00254409x
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$ OMP_NUM_THREADS=16 ./square
Thread ID: 14 out of 16 threads active
Thread ID: 4 out of 16 threads active
Thread ID: 12 out of 16 threads active
Thread ID: 6 out of 16 threads active
Thread ID: 15 out of 16 threads active
Thread ID: 1 out of 16 threads active
Thread ID: 3 out of 16 threads active
Thread ID: 11 out of 16 threads active
Thread ID: 7 out of 16 threads active
Thread ID: 8 out of 16 threads active
Thread ID: 5 out of 16 threads active
Thread ID: 9 out of 16 threads active
Thread ID: 0 out of 16 threads active
Thread ID: 13 out of 16 threads active
Thread ID: 2 out of 16 threads active
Thread ID: 10 out of 16 threads active
--- Execution Results ---
Serial Time: 1.263e-06 seconds
Parallel Time: 0.00159706 seconds
Speedup: 0.000790828x
raren@raren-Nitro-ANV16-71:~/Downloads/HPC/EXP1$

Number of Threads (p)	Serial Execution Time (seconds)	Parallel Execution Time (seconds)	Speedup (S)
2 Threads	2.546e-06	0.000295350	**0.00862005x**
4 Threads	2.564e-06	0.000439209	**0.0058367x**
8 Threads	1.686e-06	0.000662712	**0.00254409x**
16 Threads	1.263e-06	0.00159706	**0.000790828x**

3. Analysis of Results (Why is Speedup < 1?)

- **Micro-Workload Size:** The problem size is extremely small ($N = 1000$ iterations). Computing 1000 integer squares sequentially takes only **1.26 to 2.56 microseconds** (1.263e-06 s).
- **Thread Spawning & Synchronization Overhead:** Creating, spawning, and synchronizing threads across CPU cores in OpenMP requires kernel-level scheduling and context switching, which takes **295 to 1597 microseconds** (0.000295 s to 0.001597 s).
- **Amdahl's Law Observation:** When parallelization overhead (thread creation, barrier synchronization, and console I/O printing inside the parallel block) outweighs the actual computational workload, parallel execution becomes slower than serial execution. As thread count increases from 2 to 16, parallel overhead increases, resulting in lower speedup for micro-tasks.

Task 3: Calculate Theoretical Peak FLOPS of CPU

1. Mathematical Formula & Parameter Identification

Theoretical CPU floating-point performance for SIMD-based CPUs supporting **Fused Multiply-Add (FMA)** is calculated as:

$$\text{Peak FLOPS} = \text{Number of Cores} \times \text{Clock Frequency} \times \text{SIMD Width} \times 2$$

Where:

- **Number of Cores:** Physical CPU cores = **16** (Intel Core i7-14650HX)
- **Clock Frequency:** Operating frequency in Hz:
    - *Base Frequency:* 2.2 GHz ($2.2 \times 10^9 \text{ Hz}$)
    - *Max Turbo Boost Frequency (from lscpu):* 5.2 GHz ($5.2 \times 10^9 \text{ Hz}$)

- **Instruction Set & SIMD Width:** The i7-14650HX supports **AVX2** (Single Precision SIMD Width = **8** floats per instruction).
- **FMA Factor:** $1 \text{ FMA} = 2 \text{ FLOPS}$ (one multiplication + one addition per clock cycle).

2. Step-by-Step Calculation

A. Calculation at Base Frequency (2.2 GHz):

$$\text{Peak FLOPS}_{\text{Base}} = 16 \times \left(2.2 \times 10^9\right) \times 8 \times 2$$
$$\text{Peak FLOPS}_{\text{Base}} = 563.2 \times 10^9 \text{ FLOPS} = \mathbf{563.2 \text{ GFLOPS}} \ (0.563 \text{ TFLOPS})$$

B. Calculation at Max Turbo Frequency (5.2 GHz):

$$\text{Peak FLOPS}_{\text{Turbo}} = 16 \times \left(5.2 \times 10^9\right) \times 8 \times 2$$
$$\text{Peak FLOPS}_{\text{Turbo}} = 1331.2 \times 10^9 \text{ FLOPS} = \mathbf{1331.2 \text{ GFLOPS}} \ (1.331 \text{ TFLOPS})$$

3. Comparison with Manufacturer's Published FLOPS

Metric	Calculated Value (GFLOPS)	Published / Expected Benchmark Range
Base Clock Throughput (2.2 GHz)	**563.2 GFLOPS**	550 – 600 GFLOPS
Max Turbo Throughput (5.2 GHz)	**1331.2 GFLOPS (1.33 TFLOPS)**	1100 – 1250 GFLOPS

Conclusion & Architectural Analysis:

1. **Theoretical vs. Hybrid Architecture:** The theoretical peak formula assumes all 16 cores operate simultaneously at the 5.2 GHz boost frequency executing FMA instructions. In reality, the Intel Core i7-14650HX uses a hybrid architecture (8 Performance-cores + 8 Efficient-cores), where E-cores operate at lower maximum clocks (up to 3.7 GHz).
2. **Thermal & Power Limits:** Real-world benchmark performance (such as LINPACK) generally reaches **80% to 90%** of the theoretical peak due to power envelope constraints (55W TDP) and thermal regulation.
3. **Summary:** The system is capable of delivering **over 1.1 TFLOPS** of sustained single-precision compute throughput under optimized AVX2 parallel workloads.